



Gym Attendance System — Interview Cheat Sheet

Goal: Be able to confidently explain every aspect of this project in a viva/interview.

1. 🎯 Project Introduction (What & Why)

One-liner:

"I built a full-stack Gym Attendance Management System with QR-based check-in, role-based access control, and replay-attack prevention."

Problem it solves:

Traditional gyms use paper registers or manual ID-card systems for attendance. These are slow, error-prone, and insecure. Our system replaces this with a **QR code pass** displayed on the member's phone. They simply show their phone at the front desk kiosk, the QR is scanned via webcam, and attendance is logged automatically — all in under 2 seconds.

Key selling points to mention:

- Fully functional full-stack application (not a prototype)
 - Security-first design: Replay-attack prevention, bcrypt password hashing, JWT-based authentication
 - Role-based access: 3 distinct roles with different permissions (Admin, Trainer, Customer)
 - Cloud database: Neon PostgreSQL (serverless, no local DB setup needed)
 - Clean REST API with auto-generated Swagger docs
-

2. 🛠️ Tech Stack (Know Every Library & WHY You Chose It)

Backend

TECHNOLOGY	PURPOSE	WHY WE CHOSE IT
FastAPI	Web framework	Async support, auto Swagger docs at <code>/docs</code> , fast performance, built-in type validation
SQLAlchemy 2.0	ORM (Object Relational Mapper)	Most popular Python ORM, supports complex relationships, works with any SQL database
Pydantic v2	Data validation & serialization	Validates all incoming request data, auto-generates response schemas
pydantic-settings	Configuration management	Reads from <code>.env</code> files automatically, type-safe settings
python-jose	JWT token creation & verification	Standard JWT library for Python, supports HS256 algorithm
bcrypt (via passlib)	Password hashing	Industry-standard one-way hashing, resistant to brute-force attacks
psycopg2-binary	PostgreSQL driver	Required by SQLAlchemy to connect to PostgreSQL databases
uvicorn	ASGI server	High-performance production server for FastAPI apps
httpx	HTTP test client	Works with FastAPI's TestClient for writing tests
pytest	Testing framework	Industry standard for Python testing

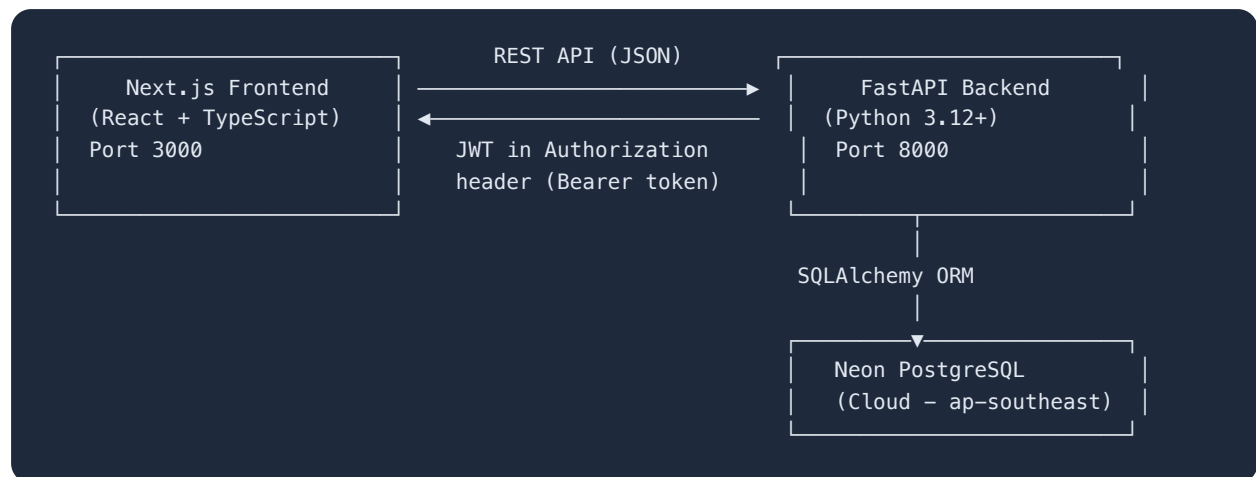
Frontend

TECHNOLOGY	PURPOSE	WHY WE CHOSE IT
Next.js 14	React framework (App Router)	Server-side rendering, file-based routing, great developer experience
React 18	UI library	Component-based architecture, hooks for state management
TypeScript	Type-safe JavaScript	Catches bugs at compile time, better IDE support
Tailwind CSS	Utility-first CSS framework	Rapid styling without writing custom CSS files
Axios	HTTP client	Better error handling than fetch, supports interceptors for auth
qrcode.react	QR code generation	Renders QR codes as SVG in React components
jsQR	QR code scanning	Pure JavaScript QR decoder — works with canvas frames from webcam
react-hot-toast	Notifications	Lightweight, good-looking toast notifications
Inter (Google Font)	Typography	Clean, modern, professional font used across the app

Database & Infrastructure

TECHNOLOGY	PURPOSE
Neon PostgreSQL	Cloud-hosted serverless PostgreSQL (no local setup, accessible anywhere)
SSL/TLS	Database connection uses <code>sslmode=require</code> for encrypted data in transit

3. 🚩 Architecture (How Everything Connects)



Communication Flow:

- Frontend sends HTTP requests via **Axios** to `http://localhost:8000/api/v1/...`
- Axios **request interceptor** automatically attaches the JWT token from localStorage to every request as `Authorization: Bearer`
- Backend validates the JWT, checks the user's role, and processes the request
- Backend queries the **Neon PostgreSQL** database via SQLAlchemy
- Response is sent back as JSON
- If a **401 Unauthorized** is received, the Axios **response interceptor** automatically clears the token and redirects to `/login`

4. 📁 Project Structure (File-by-File Explanation)

Backend (backend/app/)

```
backend/
├── .env                ← Environment variables (DB URL, JWT secret)
├── requirements.txt    ← Python dependencies
├── app/
│   ├── main.py        ← FastAPI app creation, CORS, lifespan (table creation + demo)
│   ├── core/
│   │   ├── config.py  ← Settings class (reads from .env using pydantic-settings)
│   │   └── security.py ← Password hashing (bcrypt) + JWT create/decode functions
│   ├── db/
│   │   ├── base.py    ← SQLAlchemy DeclarativeBase (shared across all models)
│   │   └── session.py  ← Engine + SessionLocal factory (connects to Neon PostgreSQL)
│   ├── models/
│   │   ├── user.py     ← User model (id, email, password, role, membership, trainer_)
│   │   ├── attendance.py ← Attendance model (check_in, check_out, marked_by)
│   │   └── token.py     ← UsedToken model (stores consumed JTIs for replay prevention)
│   ├── schemas/
│   │   ├── user.py     ← Pydantic schemas: UserCreate, UserResponse, Token, etc.
│   │   └── attendance.py ← Pydantic schemas: CheckInRequest, AttendanceLog, etc.
│   └── api/
│       ├── deps.py     ← Shared dependencies: get_db, get_current_user, RoleChecker
│       └── v1/
│           ├── router.py ← Combines all endpoint routers under /api/v1
│           └── endpoints/
│               ├── auth.py    ← /register, /login, /me
│               ├── attendance.py ← /qr-token, /verify, /manual-mark, /logs
│               └── users.py    ← /users, /{id}, /assign-trainer, /membership
```

Frontend (`frontend/src/`)

frontend/src/	
├─ middleware.ts	← Next.js route protection (checks cookie for auth)
├─ types/index.ts	← TypeScript interfaces (User, AttendanceRecord, etc.)
├─ lib/	
│ ├─ api.ts	← Axios instance + interceptors + auth helpers (login, logout)
│ └─ utils.ts	← Utility functions (formatDateTime, roleLabel, cn)
├─ components/	
│ ├─ navbar.tsx	← Top navigation bar with user info + role badge + logout
│ ├─ qr-display.tsx	← QR code generator with 30s circular countdown timer
│ ├─ qr-scanner.tsx	← Webcam-based QR scanner using jsQR + canvas
│ └─ attendance-table.tsx	← Reusable attendance history table component
└─ app/	
│ ├─ layout.tsx	← Root layout (Inter font, dark theme, Toaster provider)
│ ├─ page.tsx	← Landing page (redirects to /dashboard or /login)
│ ├─ globals.css	← Global CSS
│ └─ (auth)/login/	
│ ├─ page.tsx	← Customer login/register (tabbed: Sign In + Create Account)
│ └─ staff/page.tsx	← Staff login portal (Admin/Trainer only)
│ └─ dashboard/	
│ ├─ page.tsx	← Role-based redirector (→ /admin, /trainer, or /customer)
│ ├─ admin/page.tsx	← Admin panel: stats, attendance logs, user management, CSV export
│ ├─ trainer/page.tsx	← Trainer panel: assigned trainees, manual check-in/out
│ └─ customer/page.tsx	← Customer panel: membership card, QR pass, attendance history
└─ kiosk/page.tsx	← Public kiosk: webcam scanner + manual token input

5. 🗝️ Step-by-Step: How We Built This (Development Process)

Step 1: Database Design

- Identified 3 core entities: **User**, **Attendance**, **UsedToken**
- Chose **Neon PostgreSQL** (cloud-hosted, free tier, no local setup)
- Designed the User table with a **self-referential foreign key** (`trainer_id` → `users.id`) so a customer can be linked to their trainer

Step 2: Backend Setup (FastAPI)

- Initialized FastAPI with `pydantic-settings` for configuration management
- Created the SQLAlchemy engine and session factory pointing to Neon
- Built ORM models for User, Attendance, and UsedToken
- Created Pydantic schemas for request validation and response serialization

Step 3: Authentication System

- Implemented `/register` endpoint with **bcrypt** password hashing
- Implemented `/login` endpoint using **OAuth2PasswordRequestForm** which returns a JWT
- The JWT contains: `sub` (user ID), `role`, `jti` (unique token ID), `exp` (expiry time)

- Created `get_current_user` dependency that decodes the JWT from the `Authorization` header

Step 4: Role-Based Access Control (RBAC)

- Built a reusable `RoleChecker` class (FastAPI dependency)
- It accepts a list of allowed roles and raises `403 Forbidden` if the user's role is not in the list
- Used as: `Depends(RoleChecker([UserRole.ADMIN]))` on protected endpoints

Step 5: QR Code Pass System

- Backend: `create_qr_token()` generates a short-lived JWT (30 seconds) with a unique `jti`
- Frontend: `QRDisplay` component calls `/attendance/qr-token` every 30 seconds and renders the token as a QR code using `qrcode.react`
- Added a **circular SVG countdown timer** around the QR code for visual feedback

Step 6: QR Verification (Replay-Attack Prevention)

- Built the `/verify` endpoint: decodes the QR token, checks `jti` against the `used_tokens` table
- If `jti` already exists → **409 Conflict** (replay attack blocked!)
- If `jti` is new → store it in `used_tokens`, log attendance, return success
- This is the **most important security feature** of the project

Step 7: Frontend Development

- Set up Next.js 14 with TypeScript and Tailwind CSS
- Built separate login portals: **Customer Portal** (`/login`) and **Staff Portal** (`/login/staff`)
- Created role-based dashboards: Admin, Trainer, Customer
- Built the **Kiosk page** with a webcam-based QR scanner using `jsQR`

Step 8: Kiosk Scanner Implementation

- Uses `navigator.mediaDevices.getUserMedia()` to access the webcam
- Draws frames to a hidden `<img />` element every 200ms
- Uses `jsQR` to decode the QR from the canvas image data
- Includes **debouncing** (2.5s cooldown) to prevent duplicate scans
- Has a **mirror toggle** and **restart camera** button for usability

Step 9: Admin Features

- **User Management:** View all users with role badges and membership status
- **Staff Provisioning:** Admin can create Trainer/Admin accounts via a modal
- **Membership Management:** Admin can update a customer's membership plan (Standard/Premium/VIP) and extend it by 1/3/6/12 months
- **CSV Export:** Download attendance logs as a CSV file
- **Stats Dashboard:** Total members, today's check-ins, active trainers

Step 10: Demo Seeding

- On app startup (`lifespan`), the backend creates demo accounts if they don't exist:
- `admin@gmail.com` / `admin123` (Admin)
- `trainer@gmail.com` / `trainer123` (Trainer)
- `member@gmail.com` / `member123` (Customer)
- This makes it easy to demonstrate the project without manual setup

6. 🗝️ Security Deep-Dive (Interview Favorite!)

6.1 Password Security

- **Never store plain-text passwords!** We use `bcrypt` hashing
- `hash_password(plain)` : Generates a salt, hashes the password → stores only the hash
- `verify_password(plain, hashed)` : Re-hashes the input and compares with stored hash
- `bcrypt` is **one-way** — even if the database is compromised, passwords can't be reversed

6.2 JWT Authentication

- After login, user gets a **JWT (JSON Web Token)**
- JWT has 3 parts: `Header.Payload.Signature` (Base64 encoded)
- **Payload contains:** `sub` (user ID), `role` (ADMIN/TRAINER/CUSTOMER), `jti` (unique ID), `exp` (expiry)
- **Signed** with our `JWT_SECRET` using HS256 algorithm
- If anyone tampers with the payload, the signature won't match → token is rejected

6.3 QR Replay-Attack Prevention (★ MOST IMPORTANT)

Q: "What is a replay attack?"

A replay attack is when an attacker captures a valid QR token (e.g., by screenshot/photo) and uses it again later to check in without the real member being present.

Q: "How did you prevent it?"

1. Each QR token is a JWT with a **unique jti (JWT ID)** — a UUID generated per token
2. Tokens expire in **30 seconds** (very short window)
3. When the token is scanned and verified, the **jti** is stored in the **used_tokens** database table
4. Before processing any scan, we check: "Has this **jti** been used before?"
 - If YES → **409 Conflict** — replay attack prevented!
 - If NO → Process normally, mark **jti** as used
5. The frontend auto-rotates the QR every 30 seconds so even if someone photographs it, the old QR is already expired

6.4 RBAC (Role-Based Access Control)

- Implemented as a **reusable RoleChecker** class (a FastAPI Dependency)
- Example: `Depends(RoleChecker([UserRole.ADMIN, UserRole.TRAINER]))`
- If a Customer tries to access an admin-only route → **403 Forbidden**
- Trainers can ONLY mark attendance for their assigned trainees (checked in code)

6.5 CORS (Cross-Origin Resource Sharing)

- Backend only allows requests from `http://localhost:3000` and `http://127.0.0.1:3000`
 - Prevents malicious websites from making API calls to our backend
-

7. 🗄 Database Design (Be Ready to Draw ER Diagram)

Tables & Relationships



Key Design Decisions:

- **Self-referential FK on `users`**: `trainer_id` points back to `users.id`. This is how we link a customer to their trainer without needing a separate join table.
- **`marked_by` field**: Distinguishes if attendance was logged via QR scan or manually by a trainer. This is important for audit trails.
- **`used_tokens` table**: A security table that ensures each QR token can only be used **once**. The `expires_at` column allows periodic cleanup of old records.

8. 📡 API Endpoints (Know All 12+ Routes)

Auth Routes (`/api/v1/auth`)

ENDPOINT	METHOD	WHAT IT DOES	WHO CAN ACCESS
<code>/register</code>	POST	Creates a new user (hashes password, stores in DB)	Public
<code>/login</code>	POST	Validates credentials, returns JWT access token	Public
<code>/me</code>	GET	Returns current logged-in user's profile	Any authenticated user

Attendance Routes (`/api/v1/attendance`)

ENDPOINT	METHOD	WHAT IT DOES	WHO CAN ACCESS
<code>/qr-token</code>	GET	Generates a 30-second QR pass JWT	Customer only (+ active membership)
<code>/verify</code>	POST	Scans QR token → check-in/check-out (replay check)	Public (kiosk)
<code>/manual-mark</code>	POST	Manually mark a member present/absent	Trainer (own trainees) / Admin (any)
<code>/logs</code>	GET	Paginated attendance history (role-scoped)	All authenticated users
<code>/my-logs</code>	GET	Customer's own attendance history	Customer
<code>/report</code>	GET	Full gym-wide attendance report	Admin only

User Management Routes (/api/v1/users)

ENDPOINT	METHOD	WHAT IT DOES	WHO CAN ACCESS
/	GET	List all users (filter by role)	Admin only
/ {id}	GET	Get single user details	Admin only
/ {id}	PATCH	Update user profile/role/status	Admin only
/ {id} /assign-trainer	POST	Link a customer to a trainer	Admin only
/trainers/trainees	GET	List assigned trainees	Trainer / Admin
/ {id} /membership	PUT	Update membership plan & duration	Admin only

9. 🖥️ Frontend Pages (Know Every Screen)

PAGE	ROUTE	DESCRIPTION
Landing Page	<code>/</code>	Redirects to <code>/dashboard</code> if logged in, else to <code>/login</code>
Customer Login	<code>/login</code>	Tabbed: Sign In + Create Account (register as CUSTOMER)
Staff Login	<code>/login/staff</code>	Dedicated portal for Admin & Trainer login
Dashboard Redirector	<code>/dashboard</code>	Reads role from localStorage, redirects to role-specific dashboard
Customer Dashboard	<code>/dashboard/customer</code>	Membership card + QR pass with 30s countdown + attendance history
Trainer Dashboard	<code>/dashboard/trainer</code>	List of assigned trainees + manual check-in/out buttons + attendance table
Admin Dashboard	<code>/dashboard/admin</code>	Stats cards + tabbed view (Attendance Logs / User Management) + CSV export + Staff provisioning modal + Membership update modal
Kiosk Scanner	<code>/kiosk</code>	Full-screen webcam QR scanner + manual token paste input

10. 🔥 Hardest Challenges & How I Solved Them

Challenge 1: QR Replay-Attack Prevention

Problem: If someone takes a screenshot of the QR code, they could use it to check in without the actual member being present.

Solution: Short-lived JWTs (30s) with unique `jti` claims stored in a `used_tokens` table. Once a `jti` is used, it's permanently recorded, and any future attempt with the same `jti` is rejected with 409.

Challenge 2: Webcam QR Scanning in the Browser

Problem: The `html5-qrcode` library had compatibility issues and needed a page reload after granting camera permissions.

Solution: Switched to a **native approach** using `navigator.mediaDevices.getUserMedia()` + a hidden

`video` + `jsQR` for pure JavaScript QR decoding. This gives frame-by-frame control with no third-party camera UI dependencies.

Challenge 3: Role-Based Dashboard Routing

Problem: All users log in from the same base URL but need to see different dashboards.

Solution: After login, the user's profile (including `role`) is stored in `localStorage`. The `/dashboard` page reads the role and uses `router.replace()` to redirect to the correct role-specific dashboard. Next.js middleware also checks for auth cookies to protect routes.

Challenge 4: One Check-In & One Check-Out Per Day

Problem: A user could scan their QR multiple times in a day, creating duplicate attendance records.

Solution: On each scan, the backend queries for today's attendance record. If no record exists → create a check-in. If a record exists but no check-out → mark check-out. If both exist → reject with "already checked in and out for today."

Challenge 5: Token Storage & Auto-Logout

Problem: Where to store the JWT so it persists across page reloads but is also secure?

Solution: Dual storage — token is stored in both `localStorage` (for Axios interceptor) and a `cookie` (for Next.js middleware to read during SSR). On 401 responses, the Axios response interceptor automatically clears both and redirects to `/login`.

Challenge 6: Connecting to Neon PostgreSQL with SSL

Problem: Neon requires SSL connections. Standard PostgreSQL drivers may not enable SSL by default.

Solution: Added `?sslmode=require&channel_binding=require` to the `DATABASE_URL` in the `.env` file. Used `pool_pre_ping=True` in the SQLAlchemy engine to handle dropped cloud connections gracefully.

11. 💡 Possible Interview Questions & Answers

General

Q: What is this project about?

It's a Gym Attendance Management System where members check in using a QR code displayed on their phone, scanned by a front-desk kiosk. It supports three user roles — Admin, Trainer, and Customer — each with different permissions.

Q: Why did you choose FastAPI over Django/Flask?

FastAPI has built-in async support, automatic API documentation via Swagger, and native Pydantic integration for request/response validation. It's also significantly faster than Flask/Django for API-heavy applications.

Q: Why Next.js and not plain React?

Next.js provides file-based routing (App Router), built-in middleware for route protection, server-side rendering capabilities, and a better developer experience out of the box. For a multi-page app like this, it's a natural choice.

Security

Q: How do you store passwords?

*We use **bcrypt** hashing. When a user registers, the plain password is passed through bcrypt which generates a random salt and produces a one-way hash. We only store the hash. During login, we hash the input again and compare hashes.*

Q: What is JWT and how does it work here?

*JWT (JSON Web Token) is a compact, URL-safe token format. After login, the server creates a JWT containing the user's ID and role, signs it with a secret key using HS256, and sends it back. The client stores it and attaches it to every API request in the **Authorization: Bearer** header. The server validates the signature on each request to authenticate the user.*

Q: Can a JWT be tampered with?

If anyone modifies the payload (e.g., changes role from CUSTOMER to ADMIN), the signature will no longer match because it was signed with our secret key. The server will reject the tampered token.

Q: What happens if someone screenshots a QR code?

*Each QR code contains a JWT valid for only 30 seconds with a unique **jti**. Even if screenshotted, the QR expires in 30 seconds. And even within that 30 seconds, once it's scanned once, the **jti** is marked as used in the database, so a second scan will be rejected with a 409 Conflict error.*

Database

Q: Explain the self-referential foreign key in the User table.

The `trainer_id` column in the `users` table references back to `users.id`. This means a user with role `CUSTOMER` can be linked to another user with role `TRAINER`. SQLAlchemy handles this with `remote_side=[id]` in the relationship definition.

Q: Why use a separate `used_tokens` table instead of a blacklist?

The `used_tokens` table is specifically for QR pass JTIs (not general JWT blacklisting). We store the `jti` and `expires_at` so we can periodically clean up old tokens. It's an append-only security log.

Architecture

Q: How does RBAC work in your backend?

I built a `RoleChecker` class that acts as a FastAPI dependency. You pass it a list of allowed roles. When injected into an endpoint, it extracts the current user from the JWT and checks if their role is in the allowed list. If not, it raises a 403 Forbidden error.

Q: What is the lifespan function in main.py?

FastAPI's `lifespan` is an async context manager that runs code on app startup and shutdown. We use it to: (1) Create all database tables using `Base.metadata.create_all()`, and (2) Seed demo user accounts (admin, trainer, customer) if they don't already exist.

Q: How does the Axios interceptor work?

We have two interceptors:

1. **Request interceptor:** Before every outgoing request, it reads the JWT from `localStorage` and attaches it as `Authorization: Bearer`.
2. **Response interceptor:** If any response comes back with 401 status, it automatically clears the stored token and redirects to the login page.

Frontend

Q: How does the QR code auto-refresh work?

The `QRDisplay` component uses `setInterval` to call the `/attendance/qr-token` API every 30 seconds. Each call returns a new short-lived JWT, which is passed to `QR` (from `qrcode.react`) to render as a QR code. A separate `setInterval` drives the circular countdown animation.

Q: How does the kiosk scanner work internally?

It uses `navigator.mediaDevices.getUserMedia()` to open the webcam. The video stream is

bound to a `video` element. Every 200ms, we draw

the current video frame onto a hidden `canvas`, get the image data, and pass it to `jsQR()` which decodes any QR code found. If decoded, we call the `/verify` API. We also debounce duplicate scans within 2.5 seconds.

12. Demo Credentials (For Live Demo)

ROLE	EMAIL	PASSWORD
Admin	admin@gmail.com	admin123
Trainer	trainer@gmail.com	trainer123
Customer	member@gmail.com	member123

These are auto-seeded on server startup.

13. 🚀 How to Run the Project

```
# Backend
cd backend
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend (separate terminal)
cd frontend
npm install
npm run dev
```

- Backend: <http://localhost:8000> (API docs: <http://localhost:8000/docs>)
 - Frontend: <http://localhost:3000>
-

14. 📄 Quick Glossary (Technical Terms to Know)

TERM	MEANING
JWT	JSON Web Token — a compact, self-contained token for securely transmitting information
JTI	JWT ID — a unique identifier for each token (used for replay prevention)
bcrypt	A one-way password hashing algorithm with built-in salting
RBAC	Role-Based Access Control — restricting access based on user roles
ORM	Object Relational Mapping — maps database tables to Python classes
CORS	Cross-Origin Resource Sharing — controls which domains can access the API
ASGI	Asynchronous Server Gateway Interface — the protocol uvicorn uses to serve FastAPI
OAuth2	An authorization framework — our login uses the OAuth2 password flow
Pydantic	Python data validation library using type annotations
Interceptor	Middleware in Axios that runs before/after every HTTP request/response
Self-referential FK	A foreign key that points to the same table (users.trainer_id → users.id)
Lifespan	FastAPI startup/shutdown lifecycle hook
Neon	Serverless PostgreSQL cloud service (free tier available)

💡 **Pro Tip for the Interview:** When explaining the project, always follow this structure:

1. **What** the feature is
2. **Why** it's needed
3. **How** you implemented it (mention specific tech/files)
4. **Challenges** you faced and how you solved them